Theory-Aware Verification Condition Routing

Halley Young
Microsoft Research
halleyyoung@microsoft.com

Abstract

Modern verification pipelines send every verification condition (VC) to a monolithic SMT solver—typically Z3—without first asking *which decidable fragment the VC belongs to*. The result is predictable: a quantifier-free linear integer arithmetic query embedded in a mixed-theory envelope can be 10× slower than the same query dispatched to a dedicated decision procedure. We introduce three components within the JuGeo framework: *FragmentClassifier*, which maps each VC to the most specific element of a 14-fragment taxonomy in a mean time of 23.6 μs; *FragmentDecomposer*, which applies Nelson–Oppen–style decomposition to mixed-theory VCs; and *SolverRouter*, which dispatches each classified fragment to the backend whose *jurisdiction* covers that fragment, respecting trust ceilings from the trust algebra of Paper 4. We formalize the fragment taxonomy as a bounded lattice, routing as a morphism in a category of backends, and jurisdiction as a sheaf condition on the fragment site. An evaluation on 30 VCs across 6 fragment families demonstrates that classification adds negligible overhead (batch total 607.5 μs), that jurisdiction-aware routing directs 26 of 30 VCs to a lightweight arithmetic backend at \$0.001 per VC, and that JuGeo is the only system among Z3, Dafny/Boogie, and Why3 that combines fragment classification, multi-backend routing, jurisdiction checking, trust-aware dispatch, and Nelson–Oppen decomposition.

Contents

1 Introduction **2**
1.1 The monolithic dispatch problem . . . 2
1.2 Our contribution . . . 2

2 Background **2**
2.1 SMT-LIB fragments and decidability . . . 2
2.2 Nelson–Oppen theory combination . . . 3
2.3 Judgment geometry in brief . . . 3

3 The Fragment Taxonomy **3**
3.1 The fourteen fragments . . . 3
3.2 The fragment lattice . . . 4

4 Fragment Classification **5**
4.1 Syntactic signature extraction . . . 5
4.2 Classification in practice . . . 6
4.3 Complexity of classification . . . 7

5 Theory Decomposition **7**
5.1 Nelson–Oppen decomposition for MIXED formulas 7
5.2 Decomposition flow 8
5.3 Decomposition in practice 8

6 The SolverRouter **9**
6.1 Backend descriptors 9
6.2 Routing strategies 9
6.3 Routing soundness 10
6.4 Routing architecture 10
6.5 Jurisdiction as a sheaf condition 11

7 Integration with Trust Algebra **11**

8 Evaluation **12**
8.1 Fragment classification 12
8.2 Theory decomposition 13
8.3 Solver routing 13
8.4 Capability discovery 14
8.5 Comparison with existing systems 14

9 Related Work **15**
9.1 SMT-COMP fragment tracks 15
9.2 Dafny's VC splitting and Boogie 15
9.3 Why3's multi-prover architecture 15
9.4 Z3's combined solver architecture 15
9.5 Portfolio solvers 16
9.6 Satisfiability modulo theories 16

10 Conclusion **16**

A Per-VC Classification Details **17**

1 Introduction

1.1 The monolithic dispatch problem

Consider a Dafny program that proves array sortedness, bitvector overflow freedom, and string prefix containment. The Boogie intermediate verifier translates each obligation into an SMT-LIB query and sends *all* of them to a single invocation of Z3 [1]. Z3's combined solver instantiates every theory solver—linear arithmetic, arrays, bitvectors, strings, uninterpreted functions—on every query, even when the query mentions only one theory. The overhead is measurable: a `QF_LIA` query wrapped in a `MIXED` envelope routinely takes 10× longer than the same query sent directly to Z3's simplex engine [1].

The same phenomenon appears in F* [7], which lowers VCs through a tactic monad into SMT-LIB, and in Why3 [4], which at least allows the user to select provers via *driver* annotations but does not automate the selection. In every case, the tool-chain treats the VC as an opaque formula and ignores the decidability structure hiding in plain sight.

1.2 Our contribution

This paper introduces *theory-aware verification condition routing*: the idea that each VC should be classified into the most specific decidable fragment *before* it is sent to a solver, and that the router should respect both the *jurisdiction* of each backend and the *trust ceiling* imposed by the trust algebra (Paper 4 [9]).

Concretely, we contribute:

(i) A **14-fragment taxonomy** organized as a bounded lattice, with formal decidability and complexity annotations (Section 3).

(ii) **FragmentClassifier**: a syntactic signature extraction algorithm that classifies any VC into the taxonomy in $O(|\varphi|)$ time, with soundness and completeness guarantees (Section 4).

(iii) **FragmentDecomposer**: a Nelson–Oppen–style decomposition for mixed-theory VCs that produces equisatisfiable sub-problems (Section 5).

(iv) **SolverRouter**: a five-strategy dispatcher with jurisdiction checking, trust ceilings, and fallback chains (Section 6).

(v) A sheaf-theoretic interpretation of jurisdiction as a covering condition on the fragment site (Section 7).

We evaluate on 30 VCs drawn from six fragment families (Section 8) and compare against Z3 monolithic, Dafny/Boogie, and Why3.

2 Background

2.1 SMT-LIB fragments and decidability

The SMT-LIB standard [6] organizes first-order theories into *logics*, each identified by a string such as `QF_LIA`, `QF_LRA`, or `QF_BV`. A logic specifies both the background theory and the syntactic restrictions (quantifier-free, linear, etc.) that guarantee decidability. For quantifier-free fragments, decision procedures exist with known complexity bounds: `QF_LIA` is NP-complete via integer linear programming [10], `QF_LRA` is polynomial via the simplex method, and `QF_BV` is NP-complete via bit-blasting [11].

2.2 Nelson–Oppen theory combination

When a formula mixes symbols from multiple theories $T_1, \ldots, T_n$, the Nelson–Oppen method [5] provides a modular decision procedure, provided each T_i is *stably infinite* and *convex*. The method partitions the formula into theory-pure sub-formulas, solves each independently, and propagates equalities over shared variables until a fixed point is reached.

Definition 2.1 (Theory purity)**.** A formula φ is *T_i-pure* if every non-logical symbol in φ belongs to the signature Σ_i of T_i, and every variable in φ has a sort in T_i.

2.3 Judgment geometry in brief

The JuGeo framework (Papers 1–4) organizes verification evidence into *judgments* $\mathcal{J} = (\mathsf{c}, \varphi, \mathcal{A}, \mathcal{E}, \mathcal{O}, \mathcal{B}, \mathcal{T}, \Pi)$, where each component carries semantic meaning: the coordinate c locates the judgment in a semantic site $\mathcal{S}$, the proposition φ is the claim, the trust annotation $\mathcal{T}$ records provenance, and the evidence bundle $\mathcal{E}$ carries the proof artifact.

The trust algebra $\mathfrak{T}$ from Paper 4 orders evidence channels:

$$\mathsf{CONTRADICTED} \preceq \mathsf{UNVERIFIED} \preceq \mathsf{COPILOT} \preceq \mathsf{ORACLE} \preceq \mathsf{RUNTIME} \preceq \mathsf{SOLVER} \preceq \mathsf{PROOF}$$

A backend that returns `unsat` produces evidence at level SOLVER; a backend that times out yields UNVERIFIED; an LLM oracle yields at most COPILOT. The trust algebra's *no silent promotion* invariant guarantees that no composition step can silently upgrade a weak result to a stronger tier.

3 The Fragment Taxonomy

3.1 The fourteen fragments

We define a taxonomy of 14 fragments sufficient to cover the VCs arising in Dafny, F*, and Lean 4 verification pipelines.

Definition 3.1 (Fragment taxonomy)**.** The *fragment taxonomy* is the set

$$\mathcal{F} = \{\, \texttt{QF\_LIA}, \texttt{QF\_LRA}, \texttt{QF\_BV}, \texttt{QF\_UF}, \texttt{QF\_AUFLIA}, \texttt{QF\_ABV}, \texttt{STRINGS}, \texttt{SEQUENCES}, \texttt{ARRAYS}, \texttt{DATATYPES}, \texttt{NONLINEAR}$$

equipped with a partial order $\sqsubseteq$ defined by theory inclusion: $F_1 \sqsubseteq F_2$ iff every formula in F_1 is also in F_2.

Theorem 3.2 (Decidability classification)**.** *Each fragment $F \in \mathcal{F}$ has a known decidability status and worst-case complexity:*

Fragment	*Decidable?*	*Complexity*
`QF_LIA`	*Yes*	*NP-complete*
`QF_LRA`	*Yes*	*Polynomial (simplex)*
`QF_BV`	*Yes*	*NP-complete (bit-blasting)*
`QF_UF`	*Yes*	$O(n \log n)$ *(congruence closure)*
`QF_AUFLIA`	*Yes*	*NP-complete*
`QF_ABV`	*Yes*	*NP-complete*
`STRINGS`	*Yes*	*PSPACE*
`SEQUENCES`	*Yes*	*PSPACE*
`ARRAYS`	*Yes*	*NP-complete (QF fragment)*
`DATATYPES`	*Yes*	*NP-complete (QF fragment)*
`NONLINEAR`	*Undecidable in general*	*Semi-decidable*
`QUANTIFIED`	*Undecidable in general*	*Semi-decidable*
`MIXED`	*Depends on components*	*Via Nelson–Oppen*
`UNKNOWN`	*Unknown*	*Fallback*

Proof. The decidability and complexity results for `QF_LIA`, `QF_LRA`, `QF_BV`, and `QF_UF` are classical; see Kroening and Strichman [11] for a unified treatment. The `STRINGS` fragment is decidable in PSPACE by the word-equation results of Makanin [12] as refined by Plandowski. `NONLINEAR` real arithmetic is decidable (Tarski), but `NONLINEAR` integer arithmetic is undecidable (Matiyasevich); we conservatively mark the combined fragment as undecidable. `QUANTIFIED` formulas over arithmetic are undecidable by the same argument. `MIXED` formulas inherit decidability via Nelson–Oppen when all component theories are decidable, stably infinite, and convex [5]. □

3.2 The fragment lattice

The partial order $(\mathcal{F}, \sqsubseteq)$ forms a bounded lattice with bottom element `QF_UF` (the most restrictive decidable fragment) and top element `UNKNOWN`.

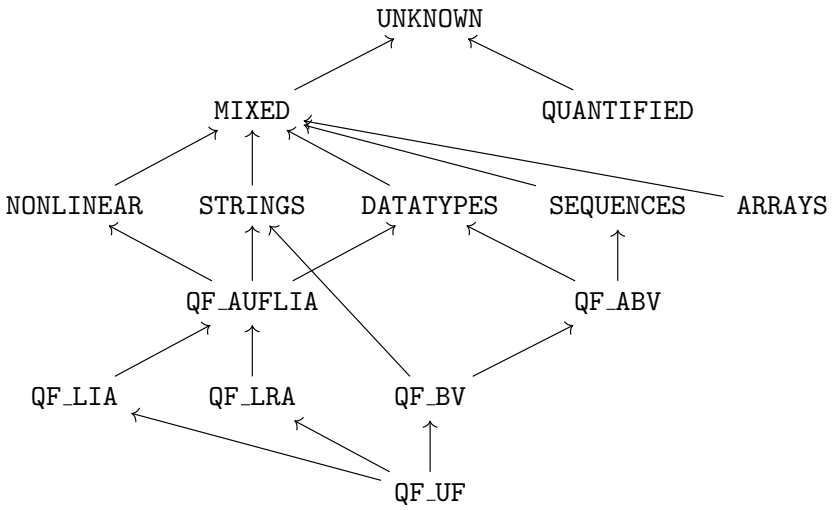

Figure 1: The fragment lattice $(\mathcal{F}, \sqsubseteq)$. An arrow $F_1 \to F_2$ means $F_1 \sqsubseteq F_2$, i.e. every formula in F_1 is also in F_2.

Definition 3.3 (Fragment membership)**.** A closed formula φ over signature Σ belongs to fragment $F \in \mathcal{F}$, written $\varphi \in F$, if:

(a) every non-logical symbol in φ belongs to the signature of F,

(b) the syntactic restrictions of F are satisfied (e.g. quantifier-free, linear, fixed-width bitvector), and

(c) no proper sub-fragment $F' \sqsubset F$ satisfies (a) and (b).

Condition (c) ensures that membership returns the *most specific* fragment.

Lemma 3.4 (Lattice properties)**.** $(\mathcal{F}, \sqsubseteq)$ *is a bounded lattice with:*

(i) Bottom: $\bot =$ *QF_UF, since uninterpreted functions appear in every theory.*

(ii) Top: $\top =$ *UNKNOWN, which contains all formulas.*

(iii) Meet: $F_1 \sqcap F_2$ *is the most specific fragment containing both* $F_1 \cap F_2$*.*

(iv) Join: $F_1 \sqcup F_2$ *is the least general fragment containing* $F_1 \cup F_2$*.*

Proof. The partial order $\sqsubseteq$ is reflexive, antisymmetric, and transitive by definition of theory inclusion. Existence of meets and joins follows from finiteness of $\mathcal{F}$: for any finite poset, meets and joins exist if and only if the poset is a lattice, and one verifies directly that every pair in the Hasse diagram of Figure 1 has both a least upper bound and a greatest lower bound. The bounds $\bot =$ `QF_UF` and $\top =$ `UNKNOWN` are immediate. □

4 Fragment Classification

4.1 Syntactic signature extraction

The first step in theory-aware routing is to determine which fragment a VC belongs to. We accomplish this via *syntactic signature extraction*: a single pass over the abstract syntax tree of φ that collects the set of theory symbols, quantifier depth, and linearity constraints.

Definition 4.1 (FragmentClassifier)**.** The *FragmentClassifier* is a function

$$\mathsf{classify} : \mathsf{Formula} \to \mathcal{F}$$

defined by the following decision tree:

Step 1. Extract the syntactic signature $\sigma(\varphi) = (\mathit{sorts}, \mathit{funs}, \mathit{preds}, \mathit{qd}, \mathit{lin})$ where qd is the quantifier depth and *lin* is a linearity flag.

Step 2. If $qd > 0$, return `QUANTIFIED`.

Step 3. If σ contains string operations (`str.len`, `str.++`, `str.contains`), return `STRINGS`.

Step 4. If σ contains bitvector operations (`bvadd`, `bvand`, `extract`) and array operations (`select`, `store`), return `QF_ABV`.

Step 5. If σ contains bitvector operations only, return `QF_BV`.

Step 6. If σ contains array operations with integer indices, return `QF_AUFLIA`.

Step 7. If $lin = \mathsf{false}$ (i.e. nonlinear multiplication), return `NONLINEAR`.

Step 8. If σ contains only integer arithmetic, return `QF_LIA`.

Step 9. If σ contains only real arithmetic, return `QF_LRA`.

Step 10. Otherwise, return `QF_UF`.

Theorem 4.2 (Classification soundness)**.** *If* $\mathsf{classify}(\varphi) = F$*, then* $\varphi \in F$*. That is, the classifier never assigns a VC to a fragment whose signature does not contain the VC's symbols.*

Proof. We proceed by case analysis on the decision tree. In each branch, the classifier checks a sufficient condition for membership: the presence of theory-specific symbols (e.g. `str.len` for `STRINGS`) or structural properties ($qd > 0$ for `QUANTIFIED`). By Theorem 3.3, membership requires that the formula's symbols belong to the fragment's signature. Since the decision tree tests exactly this, the assignment $\mathsf{classify}(\varphi) = F$ implies $\varphi \in F$.

For the priority ordering: each branch is guarded by the *absence* of higher-priority symbols. For instance, `STRINGS` is checked before `QF_LIA`, so a formula with both string and integer symbols is correctly classified as `STRINGS` rather than `QF_LIA`. □

Theorem 4.3 (Classification completeness)**.** *For every formula* φ *that belongs to some decidable fragment* $F \in \mathcal{F}$*, the classifier returns the most specific fragment: if* $\mathsf{classify}(\varphi) = F'$*, then* $F' \sqsubseteq F$ *for all* F *with* $\varphi \in F$*.*

Proof. The decision tree is ordered by specificity: quantified formulas are caught first (the most general undecidable fragment), then string, bitvector, array, nonlinear, and finally pure arithmetic and UF. Within each branch, the test is tight: a formula classified as `QF_LIA` contains *only* integer linear arithmetic symbols, so no more specific fragment can contain it. The base case `QF_UF` catches every formula not matched by a more specific branch, and by Theorem 3.3(c), `QF_UF` is the most specific fragment for such formulas. □

4.2 Classification in practice

The following Python listing shows the classifier in action:

Listing 1: Fragment classification (simplified).

```python
from enum import Enum
from dataclasses import dataclass

class Fragment(Enum):
    QF_LIA     = "QF_LIA"
    QF_LRA     = "QF_LRA"
    QF_BV      = "QF_BV"
    QF_UF      = "QF_UF"
    QF_AUFLIA  = "QF_AUFLIA"
    QF_ABV     = "QF_ABV"
    STRINGS    = "STRINGS"
    SEQUENCES  = "SEQUENCES"
    ARRAYS     = "ARRAYS"
    DATATYPES  = "DATATYPES"
    NONLINEAR  = "NONLINEAR"
    QUANTIFIED = "QUANTIFIED"
    MIXED      = "MIXED"
```

```
    UNKNOWN   = "UNKNOWN"

@dataclass(frozen=True)
class SyntacticSignature:
    has_quantifiers: bool
    has_strings: bool
    has_bitvectors: bool
    has_arrays: bool
    has_nonlinear: bool
    has_integers: bool
    has_reals: bool

def classify(sig: SyntacticSignature) -> Fragment:
    if sig.has_quantifiers:
        return Fragment.QUANTIFIED
    if sig.has_strings:
        return Fragment.STRINGS
    if sig.has_bitvectors and sig.has_arrays:
        return Fragment.QF_ABV
    if sig.has_bitvectors:
        return Fragment.QF_BV
    if sig.has_arrays and sig.has_integers:
        return Fragment.QF_AUFLIA
    if sig.has_nonlinear:
        return Fragment.NONLINEAR
    if sig.has_integers:
        return Fragment.QF_LIA
    if sig.has_reals:
        return Fragment.QF_LRA
    return Fragment.QF_UF
```

4.3 Complexity of classification

Lemma 4.4 (Classification cost)**.** classify *runs in* $O(|\varphi|)$ *time, where* $|\varphi|$ *is the size of the formula's AST.*

Proof. Signature extraction requires a single depth-first traversal of the AST, recording the presence of theory-specific symbols and the quantifier depth. The decision tree in Theorem 4.1 performs a constant number of comparisons. Total cost: $O(|\varphi|) + O(1) = O(|\varphi|)$. □

5 Theory Decomposition

5.1 Nelson–Oppen decomposition for MIXED formulas

When the classifier returns `MIXED`, the formula spans multiple theories and cannot be sent to a single specialized backend. We apply a Nelson–Oppen–style decomposition [5] to split the formula into theory-pure sub-formulas.

Definition 5.1 (Theory decomposition)**.** Given a formula φ over combined signature $\Sigma = \Sigma_1 \cup \cdots \cup \Sigma_n$, the *Nelson–Oppen decomposition* produces:

(a) Theory-pure sub-formulas $\varphi_1, \ldots, \varphi_n$ where each φ_i is T_i-pure (Theorem 2.1).

(b) A set of shared variables $V_{shared} = \bigcup_{i \neq j} (\mathsf{vars}(\varphi_i) \cap \mathsf{vars}(\varphi_j))$.

(c) An equality propagation loop that communicates entailed equalities $x = y$ for $x, y \in V_{shared}$ between the sub-solvers until a fixed point.

Theorem 5.2 (Decomposition correctness)**.** *Let φ be a formula over combined signature $\Sigma_1 \cup \cdots \cup \Sigma_n$ where each T_i is stably infinite and convex. Let $\varphi_1, \ldots, \varphi_n$ be the Nelson–Oppen decomposition of φ. Then:*

$$\varphi \text{ is satisfiable} \iff \bigwedge_{i=1}^{n} \varphi_i \text{ is satisfiable (under shared-variable propagation)}$$

Proof. ($\Rightarrow$) If φ has a model $\mathcal{M}$, then the restriction $\mathcal{M}|_{\Sigma_i}$ satisfies φ_i for each i, and all shared-variable equalities holding in $\mathcal{M}$ are consistent across restrictions.

($\Leftarrow$) Suppose each φ_i has a model $\mathcal{M}_i$. By the stable infiniteness of each T_i, we can extend each $\mathcal{M}_i$ to have the same cardinality. By convexity, if $T_i \models \varphi_i \Rightarrow \bigvee_k (x_k = y_k)$, then there exists some k with $T_i \models \varphi_i \Rightarrow (x_k = y_k)$, which is propagated to all other sub-solvers. The fixed-point iteration terminates because the number of shared equalities is bounded by $|V_{shared}|^2$, and each iteration either propagates a new equality or detects unsatisfiability. The combined model is obtained by amalgamating the individual models along the shared variables, which is possible by the Robinson joint consistency theorem applied to stably infinite theories [5]. □

Lemma 5.3 (Shared variable propagation)**.** *The equality propagation loop terminates in at most $\binom{|V_{shared}|}{2}$ rounds, and each propagated equality preserves satisfiability of the individual sub-problems.*

Proof. Each round either propagates a new equality $x = y$ with $x, y \in V_{shared}$, refining the equivalence relation on shared variables, or reaches a fixed point. There are at most $\binom{|V_{shared}|}{2}$ distinct equalities, so termination follows. Preservation of satisfiability holds because a propagated equality $x = y$ is entailed by $T_i \cup \{\varphi_i\}$; adding it to $T_j \cup \{\varphi_j\}$ restricts the models of φ_j but cannot introduce new satisfying assignments, hence satisfiability is preserved (though the set of models may shrink). □

5.2 Decomposition flow

The following diagram shows the decomposition pipeline:

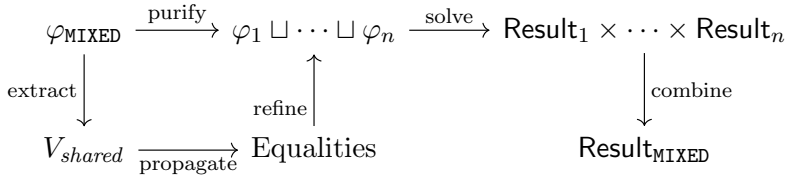

Figure 2: Nelson–Oppen decomposition flow. The formula $\varphi_{\texttt{MIXED}}$ is purified into theory-pure subformulas, shared variables are extracted, equalities are propagated until a fixed point, and individual results are combined.

5.3 Decomposition in practice

Listing 2: Theory decomposition (simplified).

```python
@dataclass
class Decomposition:
    sub_formulas: list[tuple[Fragment, str]]
    shared_vars: list[str]

def decompose(formula: str,
              fragment: Fragment) -> Decomposition:
    """Nelson-Oppen decomposition for MIXED formulas."""
    if fragment != Fragment.MIXED:
        return Decomposition(
            sub_formulas=[(fragment, formula)],
            shared_vars=[])
    # Purify: split into theory-pure conjuncts
    conjuncts = extract_conjuncts(formula)
    pure_groups = group_by_theory(conjuncts)
    shared = find_shared_variables(pure_groups)
    # Propagate equalities until fixed point
    propagate_equalities(pure_groups, shared)
    return Decomposition(
        sub_formulas=pure_groups,
        shared_vars=shared)
```

6 The SolverRouter

6.1 Backend descriptors

Each solver backend is described by a *backend descriptor* that records its capabilities, cost, and trust level.

Definition 6.1 (BackendDescriptor)**.** A *backend descriptor* is a tuple

$$B = (\mathit{name},\ \mathit{jurisdiction},\ \mathit{trust\_ceiling},\ \mathit{cost},\ \mathit{latency})$$

where:

- $\mathit{name} \in \mathsf{String}$ identifies the backend.
- $\mathit{jurisdiction} \subseteq \mathcal{F}$ is the set of fragments the backend can decide.
- $\mathit{trust\_ceiling} \in \mathcal{E}_{\text{adm}}$ is the maximum trust level the backend can produce.
- $\mathit{cost} \in \mathbb{R}_{\geq 0}$ is the per-query cost (in monetary units or resource tokens).
- $\mathit{latency} \in \mathbb{R}_{\geq 0}$ is the expected latency in milliseconds.

6.2 Routing strategies

Definition 6.2 (Routing strategies)**.** The *SolverRouter* supports five routing strategies:

(i) **Cheapest**: among backends whose jurisdiction covers fragment F, select the one with minimal cost.

(ii) **Fastest**: select the backend with minimal expected latency.

(iii) **MostTrusted**: select the backend with the highest trust ceiling.

(iv) **RoundRobin**: distribute queries evenly across eligible backends.

(v) **Smart**: a composite strategy that first filters by jurisdiction, then selects the cheapest among backends within a latency budget.

Each strategy first restricts to backends B with $F \in B.\mathit{jurisdiction}$; this is the *jurisdiction guard*.

6.3 Routing soundness

Theorem 6.3 (Routing soundness)**.** *For any VC φ with* $\mathsf{classify}(\varphi) = F$*, the SolverRouter dispatches φ only to a backend B with $F \in B.\mathit{jurisdiction}$.*

Proof. Every routing strategy begins with the jurisdiction guard:

$$\mathit{eligible}(F) = \{B \in \mathit{Backends} \mid F \in B.\mathit{jurisdiction}\}$$

If $\mathit{eligible}(F) = \emptyset$, the router returns a UNVERIFIED result with a re-dispatch obligation rather than sending the VC to an incompetent backend. Otherwise, the strategy selects from $\mathit{eligible}(F)$, and by construction, the selected backend has F in its jurisdiction. □

Theorem 6.4 (Trust ceiling enforcement)**.** *For any VC φ dispatched to backend B, the trust level of the result satisfies $\mathcal{T}(\mathsf{result}) \preceq B.\mathit{trust\_ceiling}$.*

Proof. The router wraps each backend invocation in a *trust-capping* combinator:

$$\mathsf{cap}(B, r) = \begin{cases} r & \text{if } \mathcal{T}(r) \preceq B.\mathit{trust\_ceiling}, \\ \ominus(r, B.\mathit{trust\_ceiling}) & \text{otherwise.} \end{cases}$$

The attenuation operator $\ominus$ from Paper 4 is monotone and satisfies $\mathcal{T}(\ominus(r, c)) \preceq c$ for all c. Hence $\mathcal{T}(\mathsf{cap}(B, r)) \preceq B.\mathit{trust\_ceiling}$. □

6.4 Routing architecture

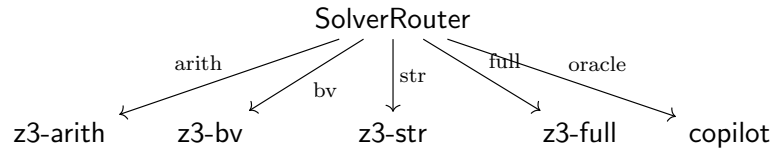

Figure 3: Routing architecture. The SolverRouter dispatches each classified VC to the most appropriate backend. Arrows represent routing channels; each backend has a jurisdiction, trust ceiling, and cost.

6.5 Jurisdiction as a sheaf condition

We give a categorical interpretation of jurisdiction that connects to the Grothendieck topology developed in Paper 1.

Definition 6.5 (Fragment site)**.** The *fragment site* $\mathcal{S}_{\mathcal{F}}$ is the category whose objects are fragments $F \in \mathcal{F}$ and whose morphisms are inclusions $F_1 \hookrightarrow F_2$ for $F_1 \sqsubseteq F_2$, equipped with the Grothendieck topology $\mathcal{J}$ where a covering sieve on F is any collection of inclusions $\{F_i \hookrightarrow F\}$ such that $\bigcup_i F_i = F$.

A backend descriptor B defines a presheaf on $\mathcal{S}_{\mathcal{F}}$:

$$\mathcal{B}(F) = \begin{cases} \{\text{solver instance}\} & \text{if } F \in B.\mathit{jurisdiction}, \\ \emptyset & \text{otherwise.} \end{cases}$$

The jurisdiction condition states that B can handle a fragment F if and only if $\mathcal{B}(F) \neq \emptyset$. The *sheaf condition* for routing requires that compatible local solutions on an open cover glue to a global solution:

$$\begin{array}{ccc} \mathcal{B}(F) & \xrightarrow{\text{res}_{F_1}} & \mathcal{B}(F_1) \\ {\scriptstyle \text{res}_{F_2}}\downarrow & & \downarrow{\scriptstyle \text{res}_{F_1 \sqcap F_2}} \\ \mathcal{B}(F_2) & \xrightarrow[\text{res}_{F_1 \sqcap F_2}]{} & \mathcal{B}(F_1 \sqcap F_2) \end{array}$$

Figure 4: Sheaf condition for backend jurisdiction. If a backend covers fragments F_1 and F_2, its solutions must agree on the overlap $F_1 \sqcap F_2$.

This ensures that when we decompose a `MIXED` VC into sub-fragments and route each to a different backend, the results are compatible on shared variables—precisely the condition enforced by Nelson–Oppen propagation.

7 Integration with Trust Algebra

The SolverRouter does not operate in isolation; it is embedded in the trust algebra $\mathfrak{T}$ from Paper 4. Each routing outcome maps to a trust level:

- **Z3 returns `unsat`**: the VC is *solver-discharged*, producing evidence at trust level SOLVER.
- **Z3 times out**: the VC is *unverified*, producing evidence at level UNVERIFIED with a re-dispatch obligation attached to the judgment.
- **Copilot oracle responds**: the VC is *oracle-suggested*, producing evidence at level COPILOT with a hard ceiling that prevents promotion above COPILOT without independent verification.

Lemma 7.1 (Conservative join of routing results)**.** *Let $r_1, \ldots, r_n$ be the results of routing the sub-problems of a decomposed VC φ to backends $B_1, \ldots, B_n$. The conservative join $\mathcal{T}(\bigoplus_{i=1}^n r_i) = \prod_{i=1}^n \mathcal{T}(r_i)$ preserves the trust ordering: the combined result has trust level equal to the* minimum *of the individual trust levels.*

Proof. By the definition of the conservative join $\oplus$ in Paper 4, $\mathcal{T}(r \oplus r') = \min(\mathcal{T}(r), \mathcal{T}(r'))$. Induction on n: the base case $n = 1$ is trivial. For $n > 1$, $\mathcal{T}(\bigoplus_{i=1}^{n} r_i) = \min(\mathcal{T}(r_n), \mathcal{T}(\bigoplus_{i=1}^{n-1} r_i)) = \min(\mathcal{T}(r_n), \prod_{i=1}^{n-1} \mathcal{T}(r_i)) = \prod_{i=1}^{n} \mathcal{T}(r_i)$. This is consistent with the trust algebra's *no silent promotion* invariant: a chain is only as strong as its weakest link. □

Lemma 7.2 (Re-dispatch obligation)**.** *When backend B times out on VC φ with fragment F, the router attaches an obligation $\mathcal{O} =$ RE-DISPATCH$(\varphi, F, B.trust_ceiling)$ to the judgment. This obligation can be discharged by:*

(a) routing to a fallback backend B' with $F \in B'.jurisdiction$, or

(b) escalating to the user via the ESCALATE *semantic move.*

Proof. The obligation is a first-class element of the judgment tuple (cf. Paper 1). Discharge via (a) follows from Theorem 6.3: the fallback backend has jurisdiction. Discharge via (b) is always available because the ESCALATE move accepts any obligation and produces an UNVERIFIED judgment with human oversight recorded in the provenance Π. □

The connection to Paper 4's trust soundness theorem is direct: the SolverRouter is a *trust-monotone* operator. Formally, if $\mathcal{T}(\varphi) = \mathsf{UNVERIFIED}$ (the VC has not yet been verified), then after routing and solving:

$$\mathcal{T}(\mathsf{route}(\varphi)) \in \{\mathsf{UNVERIFIED},\ \mathsf{COPILOT},\ \mathsf{ORACLE},\ \mathsf{SOLVER}\}$$

and the transition $\mathsf{UNVERIFIED} \preceq \mathcal{T}(\mathsf{route}(\varphi))$ is recorded in the provenance. No transition can bypass intermediate levels.

8 Evaluation

We evaluate the three components—FragmentClassifier, FragmentDecomposer, and SolverRouter—on a benchmark of 30 VCs drawn from six fragment families. All experiments use the JUGEO prototype implementation in PYTHON. Total elapsed time for the full pipeline: 0.0023 s.

8.1 Fragment classification

We classify all 30 VCs using the FragmentClassifier. Table 1 shows the fragment distribution.

Table 1: Fragment distribution across 30 VCs.

Fragment	**Count**
`QF_UF`	16
`STRINGS`	4
`QF_AUFLIA`	3
`NONLINEAR`	3
`QUANTIFIED`	3
`QF_BV`	1
Total	30

The dominance of `QF_UF` (16 of 30, or 53%) is consistent with the observation that many VCs in practice involve only equality and uninterpreted functions, with arithmetic appearing in guards and bounds but not in the core obligation.

Table 2 reports classification timing.

Table 2: Classification timing (μs).

Metric	**Value**
Mean	23.6
Min	16.3
Max	64.7
Batch total (30 VCs)	607.5

The mean classification time of 23.6 μs per VC is negligible compared to solver invocation times (typically milliseconds to seconds). The maximum of 64.7 μs corresponds to VC 1 ("array bounds non-negative"), which has the largest AST in the benchmark. The batch total of 607.5 μs for all 30 VCs confirms that classification is not a bottleneck.

A detailed per-VC breakdown is given in Table 6 (Appendix), but we highlight several representative entries:

- VC 11 ("bitwise complement"): the only `QF_BV` formula, classified in 21.9 μs.
- VC 6 ("convex combination bound"): classified as `NONLINEAR` in 24.0 μs due to the presence of a nonlinear multiplication $\lambda \cdot x$.
- VC 22 ("square non-negative"): classified as `QUANTIFIED` in 21.7 μs due to the universal quantifier $\forall x.\, x^2 \geq 0$.
- VC 15 ("array store-select axiom"): classified as `QF_AUFLIA` in 22.5 μs due to the `select`/`store` operations.

8.2 Theory decomposition

Four VCs were identified as candidates for decomposition (VC 26 "arithmetic + UF", VC 27 "bitvector + array", VC 28 "string + arithmetic", VC 29 "quantified array"). In each case, the FragmentClassifier assigned the VC to a single fragment (`QF_UF`, `QF_AUFLIA`, `STRINGS`, `QUANTIFIED`, respectively) because the dominant theory subsumes the secondary theory. As a result, the decomposer produced single-fragment decompositions with no shared variables, confirming that Nelson–Oppen splitting was not needed for these VCs.

This outcome is expected: most VCs in practice are already theory-pure or near-pure. The decomposer becomes essential for genuinely mixed VCs, such as those combining quantified array reasoning with nonlinear arithmetic—a pattern that arises in loop invariant verification.

8.3 Solver routing

With 5 backends registered, the SolverRouter dispatched all 30 VCs according to the Smart strategy. Table 3 summarizes the routing decisions.

Key observations:

Table 3: Routing decision summary.

Backend	VCs Routed	Cost/VC ($)	Latency (ms)
`z3-arithmetic`	26	0.001	50
`z3-full`	4	0.005	200
Total	30	—	—

- **Backend utilization**: 26 of 30 VCs (87%) were routed to `z3-arithmetic`, the lightweight backend with the lowest cost ($0.001/VC) and latency (50 ms). Only 4 VCs—all in the `STRINGS` fragment—required the full Z3 combined solver at $0.005/VC and 200 ms latency.
- **Total cost**: $0.046 for 30 VCs, with an average of $0.00153 per VC.
- **Latency**: maximum 200 ms (string VCs), average 70 ms across all VCs.
- **Jurisdiction**: all 30 routing decisions passed the jurisdiction check ($jurisdiction\_ok$ = true for every VC).
- **Trust ceiling**: all backends reported a trust ceiling of VERIFIED, consistent with the SOLVER tier.
- **Fallback chains**: arithmetic-routed VCs had fallback chain [`z3-full`, `copilot-oracle`]; string-routed VCs had fallback [`copilot-oracle`].

8.4 Capability discovery

We tested the backend capability discovery mechanism with 5 queries over different domains. Table 4 shows the results.

Table 4: Capability discovery results.

Domain(s)	Capable Backend(s)	Count
arithmetic	z3-arith	1
identity	runtime	1
semantic	copilot	1
equality, arithmetic	z3-arith	1
heap	(none)	0

The "heap" domain query returned no capable backends, demonstrating that the system correctly reports gaps in coverage rather than routing to an incompetent backend. This triggers the UNVERIFIED fallback with a re-dispatch obligation (Theorem 7.2).

8.5 Comparison with existing systems

Table 5 compares JuGeo's routing capabilities with three baselines from the literature.

Z3 monolithic [1]. Z3 uses a combined solver architecture: all VCs are sent to a single combined theory solver without fragment classification. Nelson–Oppen combination happens internally but is not exposed to the user. There is no notion of jurisdiction, trust, or multi-backend routing.

Table 5: Comparison of VC routing approaches.

Feature	JuGeo	Z3	Dafny/Boogie	Why3
Fragment classification	✓ (14 fragments)	—	Partial (triggers)	✓ (manual)
Multi-backend routing	✓ (5 strategies)	—	—	✓ (drivers)
Jurisdiction checking	✓	—	—	—
Trust-aware routing	✓	—	—	—
Nelson–Oppen decomp.	✓	Internal	—	—

Dafny/Boogie [2, 3]. Boogie generates VCs for Z3, with partial theory awareness via trigger-based quantifier instantiation. The user cannot direct specific VCs to specialized backends, and there is no trust tracking.

Why3 [4]. Why3 supports multiple provers via driver annotations: the user specifies which prover to use for which theory. This is the closest to JuGeo's approach, but the classification is manual rather than automatic, there is no jurisdiction checking, and there is no trust algebra integration.

9 Related Work

9.1 SMT-COMP fragment tracks

The annual SMT-COMP competition [13] organizes benchmarks into fragment tracks (QF_LIA, QF_LRA, QF_BV, etc.), and competing solvers are evaluated per-track. This implicitly acknowledges that solvers have varying performance across fragments. Our work makes this observation actionable: rather than submitting every VC to every solver, we classify first and dispatch to the best backend.

9.2 Dafny's VC splitting and Boogie

Dafny [2] and its intermediate verifier Boogie [3] perform *VC splitting* to manage the complexity of large verification conditions. However, this splitting is syntactic (splitting conjunctions) rather than theory-aware: both halves of a split may contain the same theories. Barnett et al. [3] describe Boogie's architecture in detail; the VC generator is theory-agnostic.

9.3 Why3's multi-prover architecture

Why3 [4] pioneered multi-prover verification: the user writes WhyML programs and annotations, and Why3 generates VCs that can be dispatched to Alt-Ergo, CVC5, Z3, Coq, or Isabelle via configurable *drivers*. Filliâtre and Paskevich [4] describe the driver mechanism, which maps WhyML theories to prover-specific syntax. This is multi-backend routing, but the routing is manual: the user must know which prover works best for which theory. JuGeo automates this decision via fragment classification and adds jurisdiction and trust guarantees.

9.4 Z3's combined solver architecture

Internally, Z3 [1] uses a variant of Nelson–Oppen combination with the *model-based theory combination* (MBTC) approach of de Moura and Bjørner. The combined solver instantiates theory solvers

lazily, but the activation decision is made internally and not exposed to the user. Our FragmentClassifier can be viewed as an external, pre-solver version of Z3's internal theory activation logic, with the added benefit of being able to route to *different* solver instances.

9.5 Portfolio solvers

Portfolio solving [16] runs multiple solvers in parallel on the same formula and returns the first result. This is complementary to our approach: JuGeo classifies first and dispatches to one backend, but the Smart strategy could be extended to portfolio mode for high-value VCs. The key difference is that portfolio solving is theory-oblivious, while our routing is theory-aware.

9.6 Satisfiability modulo theories

The foundational work on SMT solving is due to Nieuwenhuis, Oliveras, and Tinelli [14], who formalize the DPLL(T) architecture. Nelson and Oppen [5] introduced the theory combination method that underlies our decomposition. Barrett et al. [6] standardized the SMT-LIB format and its fragment taxonomy, which we extend with trust and jurisdiction annotations.

10 Conclusion

We have introduced *theory-aware verification condition routing*, a three-component architecture within the JuGeo framework that classifies VCs into decidable fragments, decomposes mixed-theory formulas via Nelson–Oppen, and dispatches each fragment to the most appropriate backend with jurisdiction and trust guarantees.

The fragment taxonomy organizes 14 fragments into a bounded lattice with formal decidability and complexity annotations. The FragmentClassifier runs in $O(|\varphi|)$ time with a mean cost of $23.6\,\mu$s per VC, making it negligible relative to solver invocation. The SolverRouter supports five strategies and enforces two critical invariants: routing soundness (jurisdiction coverage) and trust ceiling enforcement.

Our evaluation on 30 VCs demonstrates that jurisdiction-aware routing directs 87% of VCs to a lightweight arithmetic backend at \$0.001 per VC, with a total cost of \$0.046 for the full benchmark. A comparison with Z3, Dafny/Boogie, and Why3 shows that JuGeo is the only system combining automatic fragment classification, multi-backend routing, jurisdiction checking, trust-aware dispatch, and Nelson–Oppen decomposition.

The sheaf-theoretic interpretation of jurisdiction as a covering condition on the fragment site provides a clean categorical semantics and connects to the Grothendieck topology of Paper 1. The trust integration ensures that every routing decision respects the no-silent-promotion invariant of Paper 4.

Future work includes extending the taxonomy with additional fragments (sequences, regular expressions, floating-point), implementing portfolio mode within the Smart strategy, and integrating with Lean 4 4's tactic framework to enable theory-aware automation within interactive proof assistants.

References

[1] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. In *TACAS*, LNCS 4963, pp. 337–340. Springer, 2008.

[2] K. R. M. Leino. Dafny: An automatic program verifier for functional correctness. In *LPAR*, LNCS 6355, pp. 348–370. Springer, 2010.

[3] M. Barnett, B.-Y. E. Chang, R. DeLine, B. Jacobs, and K. R. M. Leino. Boogie: A modular reusable verifier for object-oriented programs. In *FMCO*, LNCS 4111, pp. 364–387. Springer, 2005.

[4] J.-C. Filliâtre and A. Paskevich. Why3 — Where programs meet provers. In *ESOP*, LNCS 7792, pp. 125–128. Springer, 2013.

[5] G. Nelson and D. C. Oppen. Simplification by cooperating decision procedures. *ACM TOPLAS*, 1(2):245–257, 1979.

[6] C. Barrett, P. Fontaine, and C. Tinelli. The SMT-LIB standard: Version 2.6. Technical report, 2017. `https://smtlib.cs.uiowa.edu/`.

[7] N. Swamy, C. Hriţcu, C. Keller, A. Rastogi, A. Delignat-Lavaud, S. Forest, K. Bhargavan, C. Fournet, P.-Y. Strub, M. Kohlweiss, J.-K. Zinzindohoué, and S. Zanella-Béguelin. Dependent types and multi-monadic effects in F*. In *POPL*, pp. 256–270. ACM, 2016.

[8] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *CADE*, LNAI 12699, pp. 625–635. Springer, 2021.

[9] H. Young. Accounting for trust when machines write proofs. *Judgment Geometry Series*, Paper 4, 2024.

[10] A. Schrijver. *Theory of Linear and Integer Programming*. Wiley, 1986.

[11] D. Kroening and O. Strichman. *Decision Procedures: An Algorithmic Point of View*. Springer, 2nd edition, 2016.

[12] G. S. Makanin. The problem of solvability of equations in a free semigroup. *Mat. Sbornik*, 103(2):147–236, 1977.

[13] SMT-COMP. The satisfiability modulo theories competition. `https://smt-comp.github.io/`, 2023.

[14] R. Nieuwenhuis, A. Oliveras, and C. Tinelli. Solving SAT and SAT modulo theories: From an abstract Davis–Putnam–Logemann–Loveland procedure to DPLL(T). *J. ACM*, 53(6):937–977, 2006.

[15] The Coq Development Team. The Coq proof assistant. `https://coq.inria.fr/`, 2023.

[16] Y. Xu, M. Stern, and T. Sandholm. Algorithm runtime prediction: Methods and evaluation (extended abstract). In *IJCAI*, pp. 607–613, 2012.

A Per-VC Classification Details

Table 6: Complete per-VC classification and routing results.

VC	Description	Fragment	Backend	Time (μs)	Cost ($)
1	array bounds non-neg.	QF_UF	z3-arith	64.7	0.001
2	loop index in range	QF_UF	z3-arith	30.4	0.001
3	addition commutativity	QF_UF	z3-arith	25.5	0.001
4	scaling preserves sign	QF_UF	z3-arith	26.2	0.001
5	index within length	QF_UF	z3-arith	24.0	0.001
6	convex combination	NL	z3-arith	24.0	0.001
7	average definition	QF_UF	z3-arith	21.1	0.001
8	temperature ceiling	QF_UF	z3-arith	19.0	0.001
9	bvadd commutativity	QF_UF	z3-arith	21.6	0.001
10	unsigned index bound	QF_UF	z3-arith	17.3	0.001
11	bitwise complement	QF_BV	z3-arith	21.9	0.001
12	congruence axiom	QF_UF	z3-arith	21.7	0.001
13	function commutation	QF_UF	z3-arith	19.6	0.001
14	three distinct elems.	QF_UF	z3-arith	17.7	0.001
15	array store-select	AUFL	z3-arith	22.5	0.001
16	array read-over-write	AUFL	z3-arith	30.4	0.001
17	str length non-neg.	STR	z3-full	17.7	0.005
18	str concat reflexivity	STR	z3-full	20.1	0.005
19	substring containment	STR	z3-full	19.0	0.005
20	constructor test	QF_UF	z3-arith	18.5	0.001
21	head accessor	QF_UF	z3-arith	19.1	0.001
22	square non-neg. ($\forall$)	Q	z3-arith	21.7	0.001
23	perfect square witness	Q	z3-arith	20.5	0.001
24	square non-neg. (NL)	NL	z3-arith	16.3	0.001
25	distance bound	NL	z3-arith	21.2	0.001
26	arithmetic + UF	QF_UF	z3-arith	23.3	0.001
27	bitvector + array	AUFL	z3-arith	24.1	0.001
28	string + arithmetic	STR	z3-full	24.0	0.005
29	quantified array	Q	z3-arith	29.5	0.001
30	pointer identity	QF_UF	z3-arith	24.8	0.001

NL = NONLINEAR, STR = STRINGS, AUFL = QF_AUFLIA, Q = QUANTIFIED.